UC San Diego

DSC 102
Systems for Scalable Analytics

Alexander Kolovos

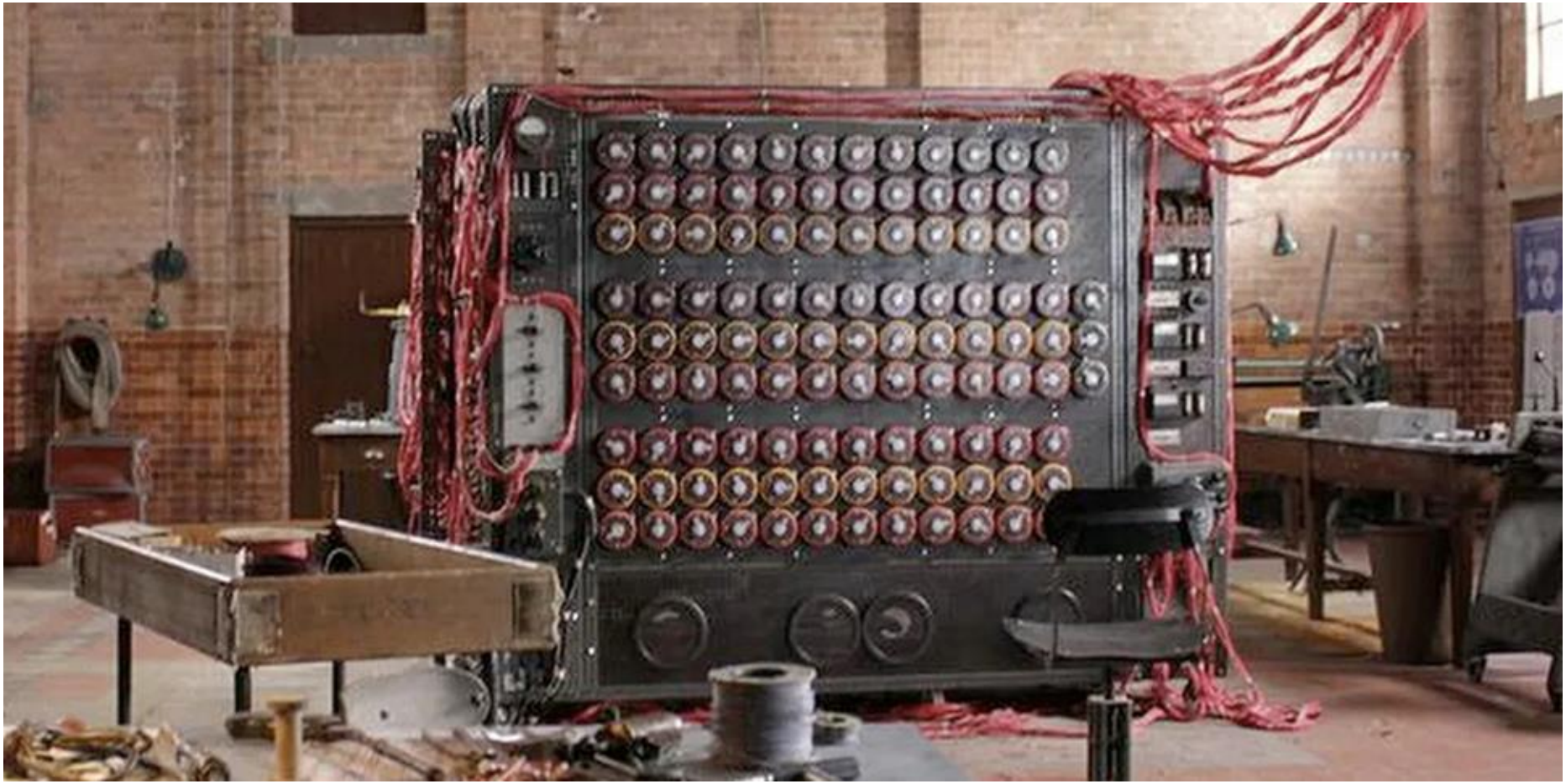
Topic 1: Basics of Machine Resources
Part 1: Computer Organization

Ch. 1, 2.1-2.3, 2.12, 4.1, and 5.1-5.5 of CompOrg Book

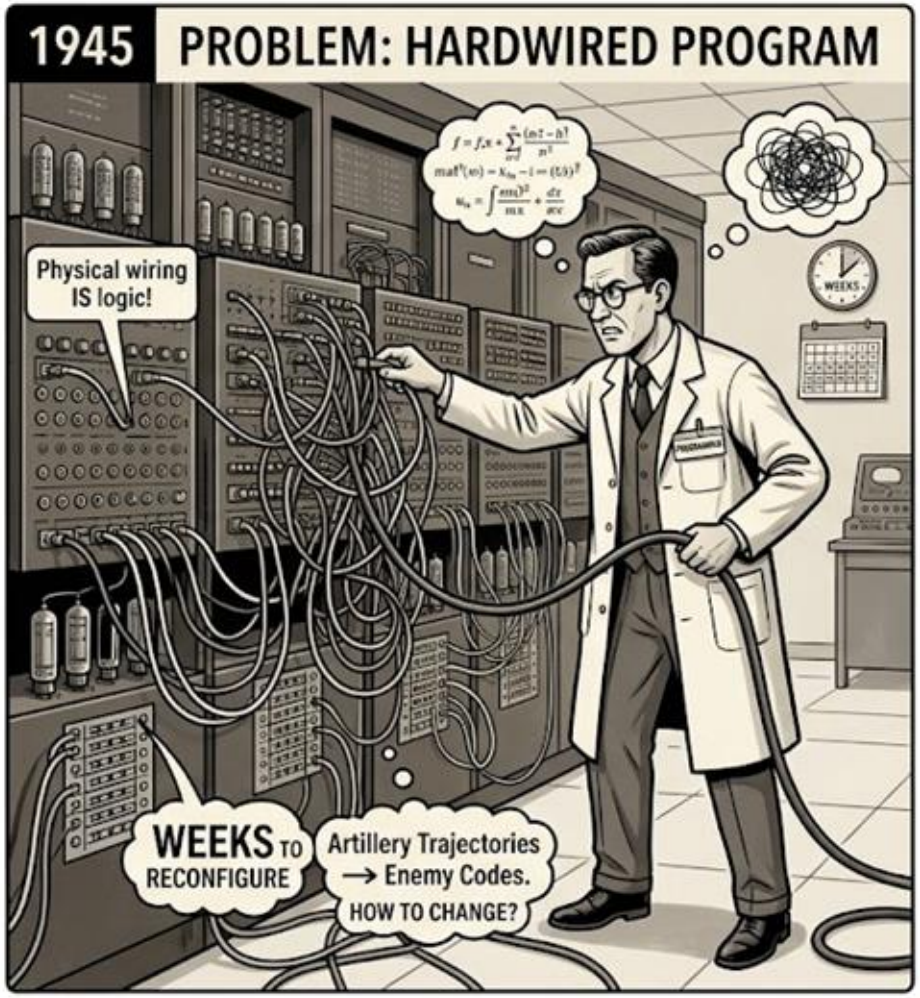
Topic 1 – Outline

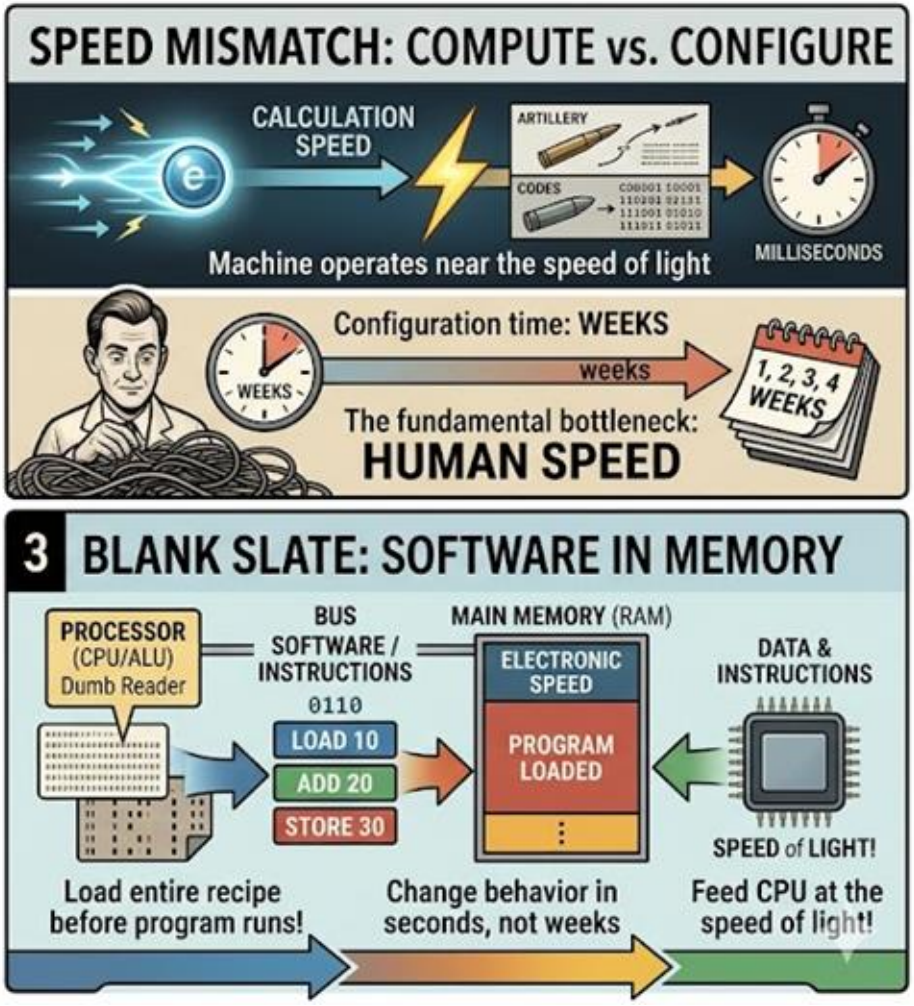
- Basics of Computer Organization
  - Digital Representation of Data
  - Processors and Memory Hierarchy
- Basics of Operating Systems
  - Process Management: Virtualization; Concurrency
  - Filesystem and Data Files
  - Main Memory Management
- Persistent Data Storage

Recognize This?



Changing The Problem





Q: *What is a computer?*

A programmable electronic device that can store, retrieve, and process digital data.

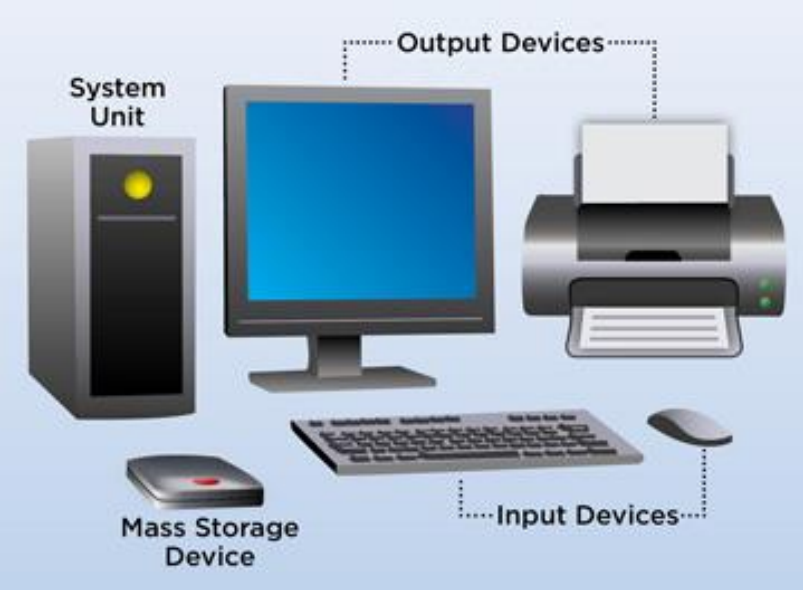
John Mauchly and Presper Eckert
(inventors of the Electronic Numerical Integrator and Computer or ENIAC, 1943-45).

Peter Naur (1928-2016)
Computer science pioneer; proposed alternative term “Datalogy” (still used in his native country Denmark) or “Data Science”.

Parts of a Computer

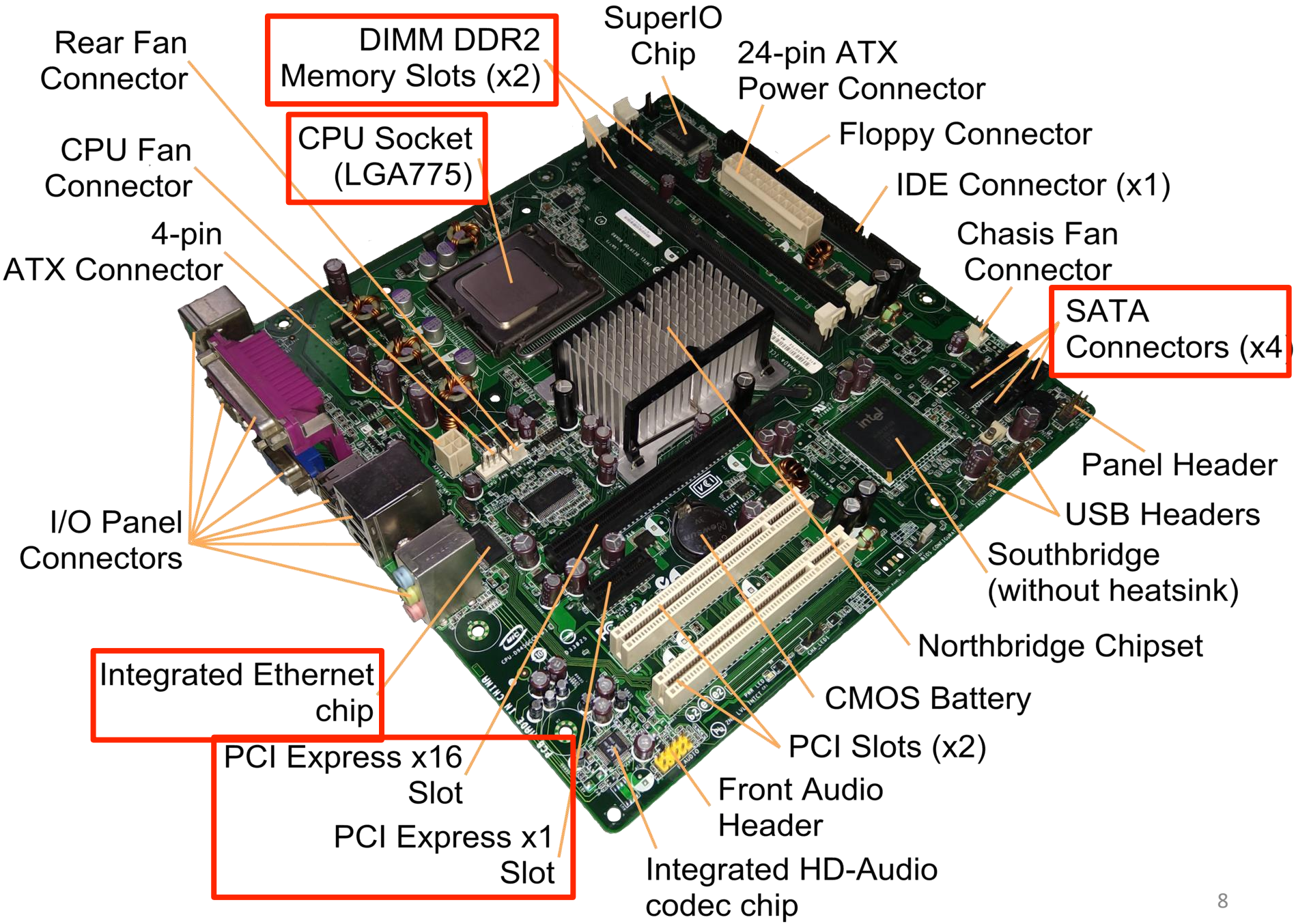
Hardware:
The electronic machinery (wires, circuits, transistors, capacitors, devices, etc.)

Software:
Programs (instructions) and data



Key Parts of Computer Hardware

- **Processor** (CPU, GPU, etc.)
  - Hardware to orchestrate and execute *instructions* to manipulate *data* as specified by a *program*
- **Main Memory** (aka Dynamic Random Access Memory)
  - Hardware to store *data* and *programs* that allows very fast location/retrieval; byte-level *addressing* scheme
- **Storage** (aka secondary/persistent storage)
  - Similar to memory but *persistent*, *slower*, and higher capacity / cost ratio; various addressing schemes
- **PCIe Bus** – for GPUs, SSDs and other external devices
- **Network** interface controller (NIC)
  - Hardware to send data to / retrieve data over network of interconnected computers/devices


Rear Fan Connector
DIMM DDR2 Memory Slots (x2)
SuperIO Chip
24-pin ATX Power Connector
CPU Socket (LGA775)
Floppy Connector
CPU Fan Connector
IDE Connector (x1)
4-pin ATX Connector
Chasis Fan Connector
SATA Connectors (x4)
Panel Header
USB Headers
I/O Panel Connectors
Southbridge (without heatsink)
Northbridge Chipset
Integrated Ethernet chip
CMOS Battery
PCI Slots (x2)
PCI Express x16 Slot
Front Audio Header
PCI Express x1 Slot
Integrated HD-Audio codec chip

Key Aspects of Software

- **Instruction**
  - A command understood by hardware; finite vocabulary for a processor: Instruction Set Architecture (ISA); bridge between hardware and software
- **Data**
  - Digital representation of *information* that is stored, processed, displayed, retrieved, or sent by a program
- **Program** (aka **code**)
  - A collection of instructions for hardware to execute
- **Programming Language (PL)**
  - A human-readable *formal* language to write programs; at a much higher level of *abstraction* than ISA
- **Application Programming Interface (API)**
  - A set of functions (“interface”) exposed by a program/set of programs for use by humans/other programs

Kinds of Software

- **Firmware**
  - Read-only programs “baked into” a device to offer basic hardware control functionalities
- **Operating System (OS)**
  - Collection of interrelated programs that work as an intermediary platform/service to enable application software to use hardware more effectively/easily
  - Examples: Linux, Windows, MacOS, etc.
- **Application Software**
  - A program or a collection of interrelated programs to manipulate data, typically designed for human use
  - Examples: Excel, Chrome, PostgreSQL, etc.

How Does It All Come Together?

Programming Models

- How does an assembly program end up executing as digital logic?
- What happens in-between?
- How is a computer designed using logic gates and wires to satisfy specific goals?

Architecture

"C" as a model of computation

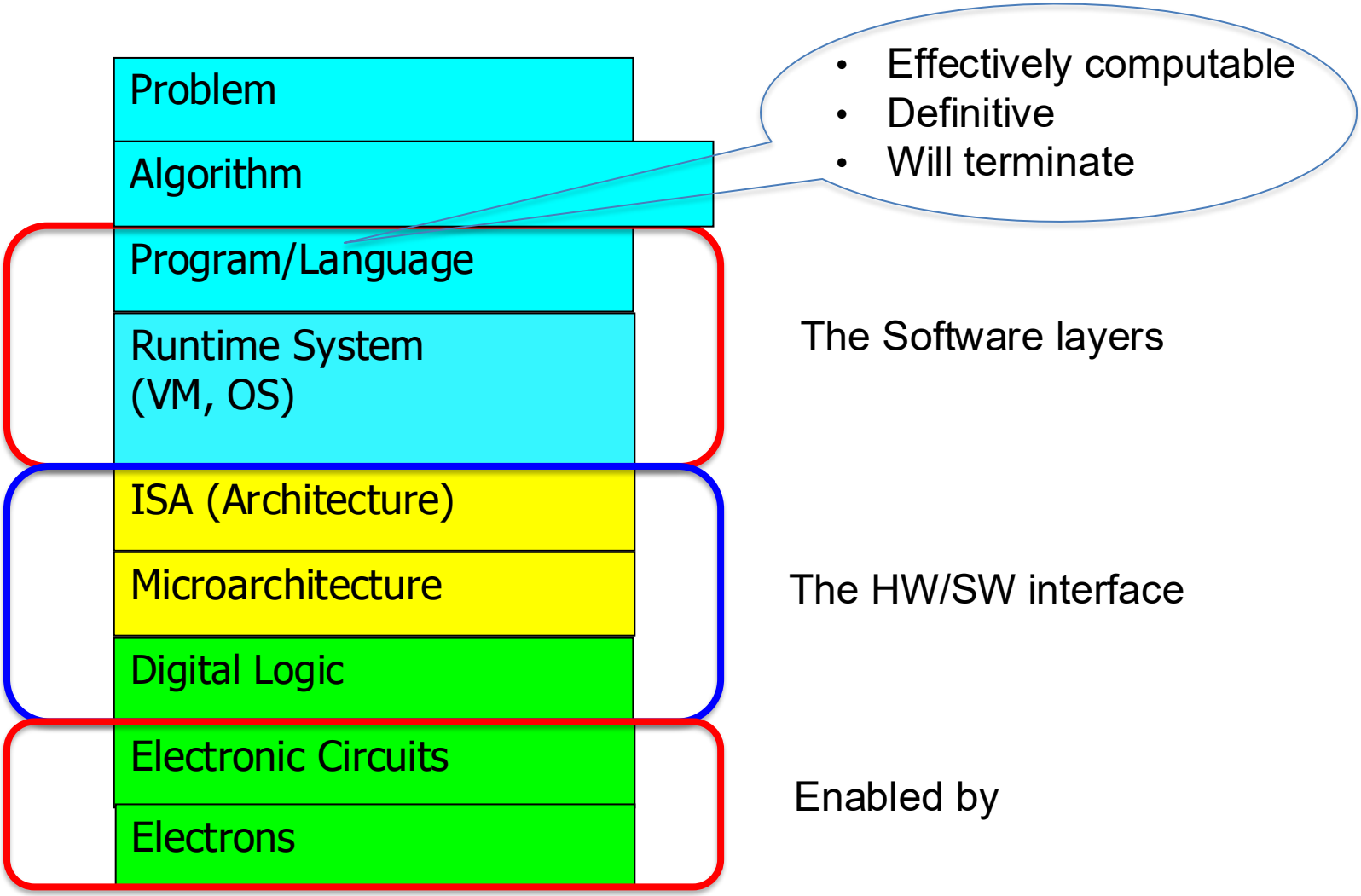
Programmer's view of a computer system works

Architect/microarchitect's view: How to design a computer that meets system design goals. Choices critically affect both the SW programmer and the HW designer

HW designer's view of a computer system works

Digital logic as a model of computation

Transformations And Abstraction



The Power Of Abstraction

Levels of transformation create abstractions

- *Abstraction*: A higher level only needs to know about the interface to the lower level, not how the lower level is implemented

  E.g., high-level language programmer does not really need to know what the ISA is and how a computer executes instructions

Abstraction improves productivity

- No need to worry about decisions made in underlying levels

  E.g., programming in Java vs. C vs. assembly vs. binary vs. by specifying control signals of each transistor every cycle

Transformations – Bubble Sort

- The Algorithm
  - Start from the beginning of an array, and compare each pair of adjacent elements
    - If elements in pair are in reversed order, then swap them.
  - If at least one swap has been done, repeat initial step.

A C++ Realization Of Bubble Sort

```
void bubbleSort(int arr[], int n) {
      bool swapped = true;
      int j = 0;
      int tmp;
      while (swapped) {
            swapped = false;
            j++;
            for (int i = 0; i < n - j; i++) {
                  if (arr[i] > arr[i + 1]) {
                        tmp = arr[i];
                        arr[i] = arr[i + 1];
                        arr[i + 1] = tmp;
                        swapped = true;
                  }
            }
      }
}
```

ARM Assembly - 1

```
.equ datum_size,4                    @ works with 4 byte (32 bit) words
.globl _bubble
.text
_bubble:
 stmfd sp!, {r4, r5, r6, r7, r8, lr}   @ save variables to stack
 cmp r1, #1                          @ number of elements must be > 1
 ble end_outer                       @ stop if nothing to do

 sub r5, r1, #1                      @ need n-1 comparisons
 mov r4, r0                          @ initialise current pointer
 mov r6, #0                          @ this register set when we swap

loop_start:
 ldr r7, [r4], #datum_size           @ load one element
 ldr r8, [r4]                        @ and next one
 cmp r7, r8                          @ compare them
 ble no_swap                         @ branch if second greater
```

ARM Assembly - 2

```
mov r6, #1                      @ set keep_going flag
 sub r4, r4, #datum_size        @ reset pointer to first element
 swp r8, r8, [r4]               @ exchange value in r8 and address in r4
 str r8, [r4, #datum_size]!     @ store new r8 to incremented address
no_swap:
 subs r5, r5, #1                @ decrement counter
 bne loop_start                 @ and restart loop if more needed

end_inner:
 cmp r6, #0                     @ check keep_going flag
 beq end_outer                  @ and leave if not set

 mov r6, #0                     @ clear keep_going flag
 mov r4, r0                     @ reset pointer
 sub r5, r1, #1                 @ reset counter
 b loop_start                   @ start another iteration

end_outer:
        ldmfd   sp!, {r4, r5, r6, r7, r8, pc}
                        @ restore state from stack and leave subroutine
```

The Stored Program Epiphany

0100 1000 0110 1001 | 0100 1000 0110 1001

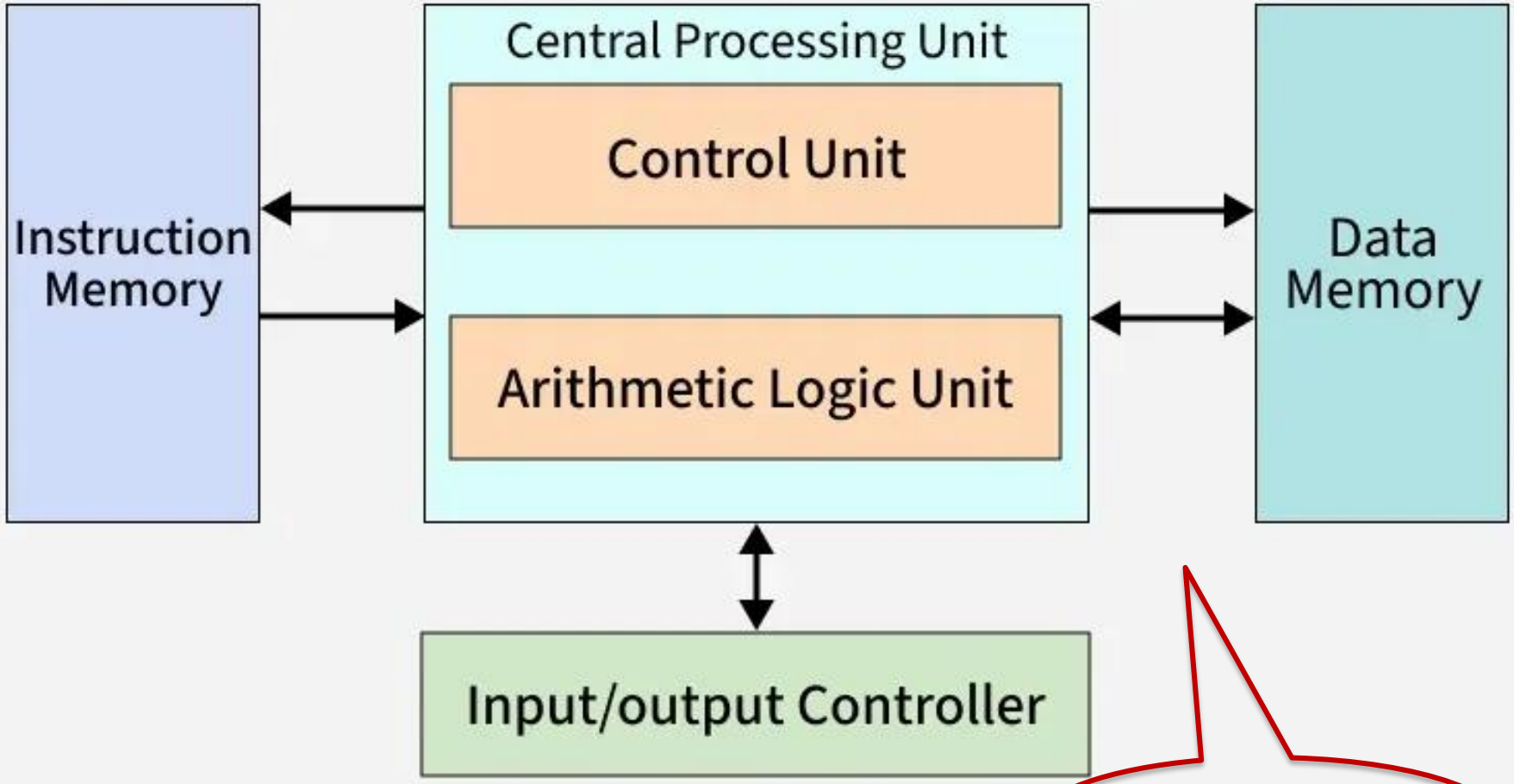
You are looking at two identical sequences of electrons trapped in a silicon chip.

Sequence A is a piece of Data—it is the exact 16-bit integer for the number 18,537.

Sequence B is a piece of Software—it is the literal ARM Assembly machine code instruction to ADD two registers together.

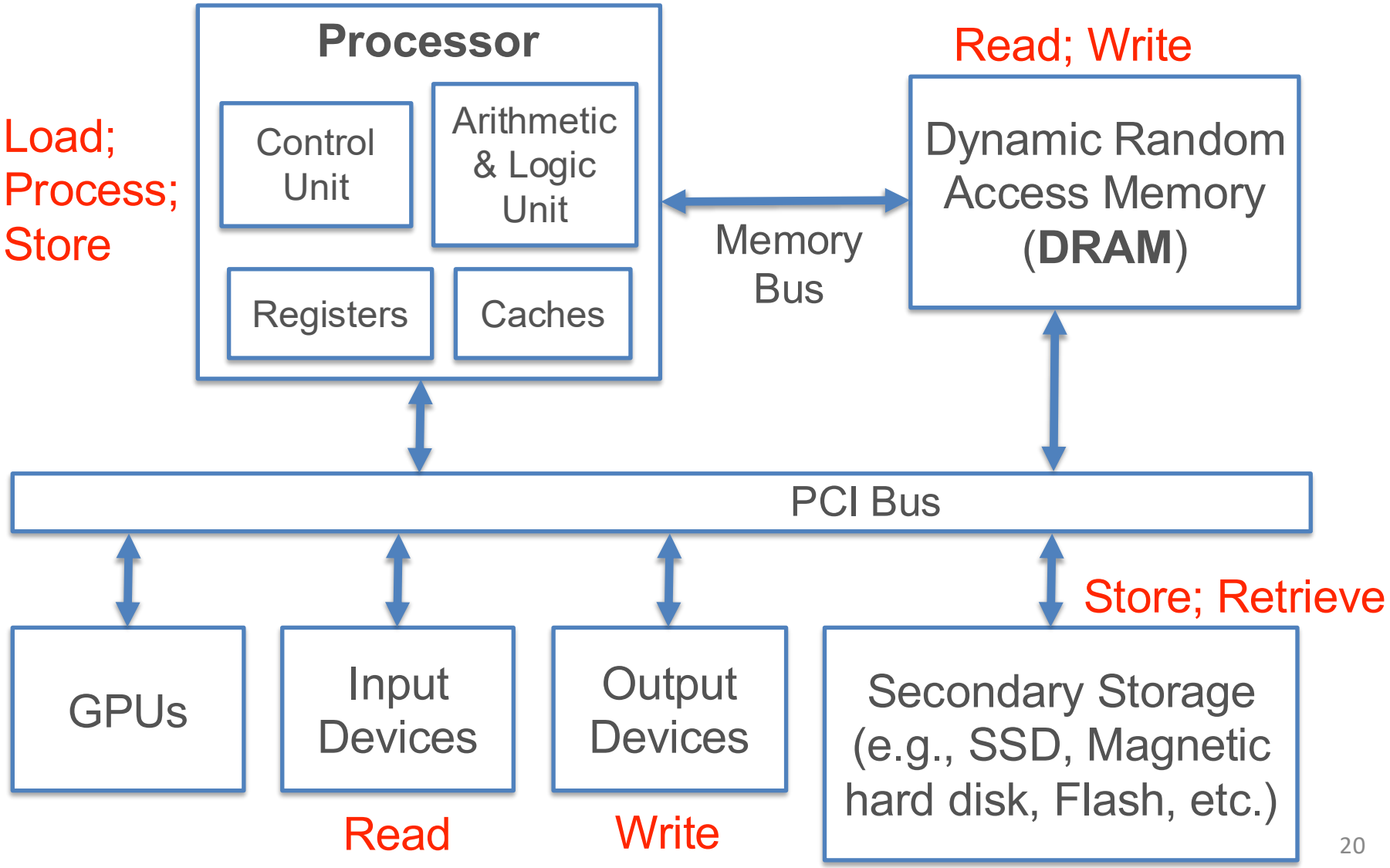
The Task: If you were a hardware engineer in 1945 tasked with building the physical memory banks for this computer, how do you design a physical copper wire that 'knows' how to carry the math instruction differently than the number? How do you keep them separate?

A Way Out?

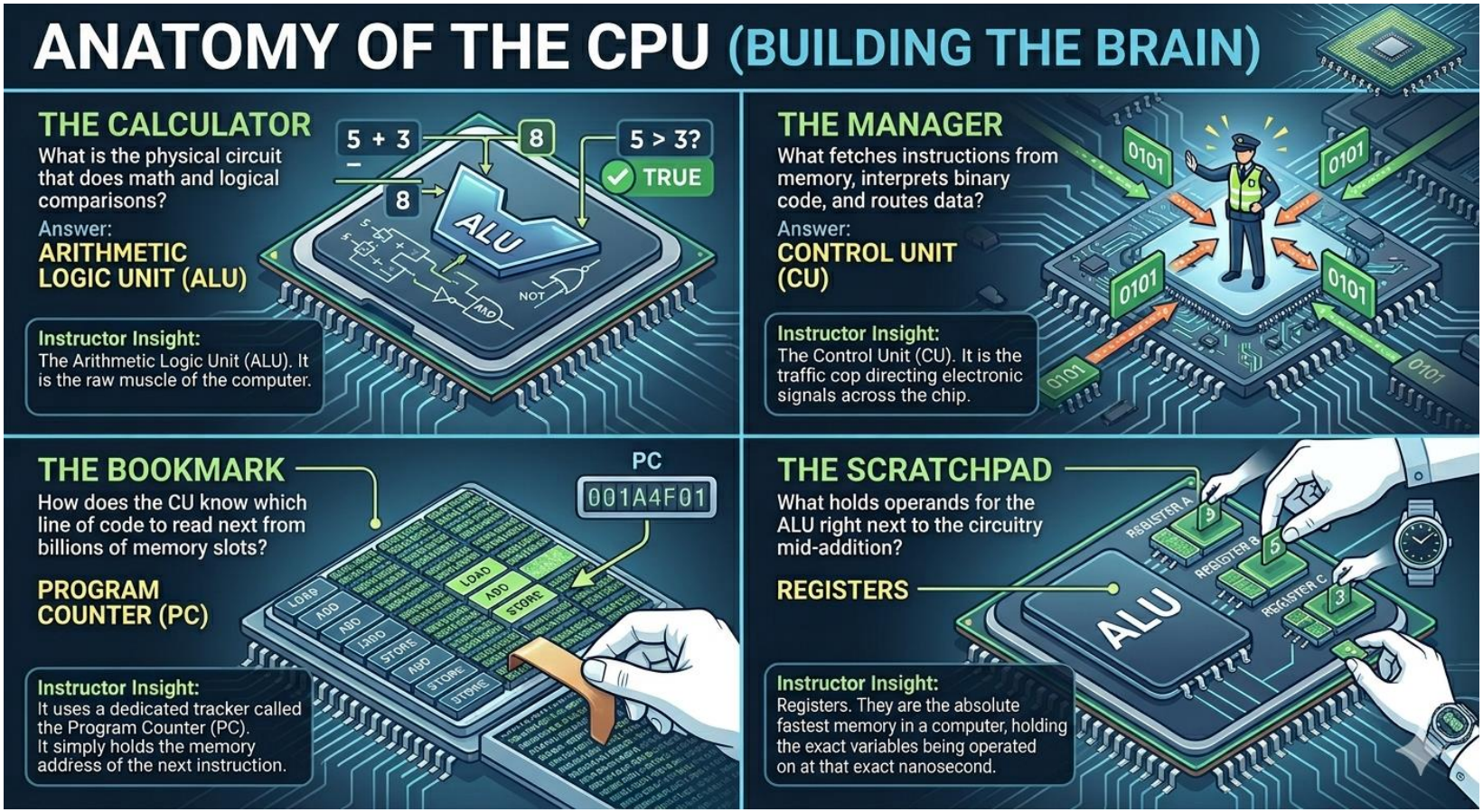


Harvard Architecture

Modern Computers



Visualizing The Functions



Computer: The Von Neumann Model

Also called *stored program computer* (instructions in memory). Two key properties:

- **Stored program**
  - Instructions stored in a linear memory array
  - Memory is unified between instructions and data
    - The interpretation of a stored value depends on the control signals
- **Sequential instruction processing**
  - One instruction processed (fetched, executed, and completed) at a time
  - Program counter (instruction pointer) identifies the current instruction.
  - Program counter is advanced sequentially except for control transfer instructions

Computer: Von Neumann vs. Dataflow

- **Von Neumann model**:
  An instruction is fetched and executed in control flow order
  - As specified by the instruction pointer
  - Sequential unless explicit control flow instruction (e.g., branch)

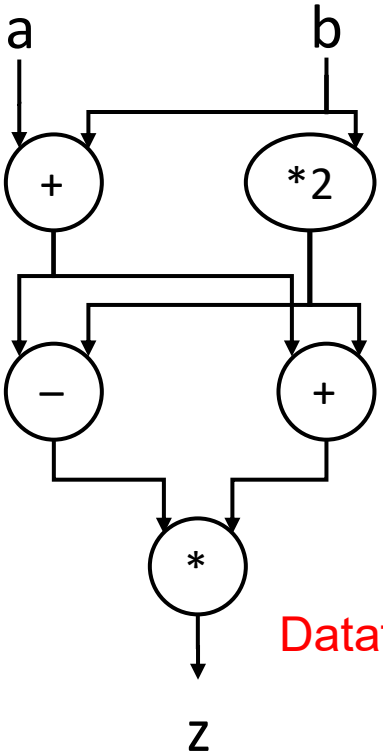
- **Dataflow model**:
  An instruction is fetched and executed in data flow order
  - i.e., when its operands are ready
  - i.e., there is no instruction pointer
  - Instruction ordering specified by data flow dependence
    - Each instruction specifies “who” should receive the result
    - An instruction can “fire” whenever all operands are received
  - Potentially many instructions can execute at the same time
    - Inherently more parallel

Computer: Von Neumann vs. Dataflow

- Consider a von Neumann program
  - What is the significance of the program order?

V = a + b;
W = b * 2;
X = v - w
Y = v + w
Z = x * y

Sequential



Dataflow

- Which model is more natural to you as a programmer?

Basics of Processors

Q: *How does a processor execute machine code?*

From Programs To Machine Code

C:

```
car *c = malloc(sizeof(car));
c->miles = 100;
c->gals = 17;
float mpg = get_mpg(c);
free(c);
```

Java:

```
Car c = new Car();
c.setMiles(100);
c.setGals(17);
float mpg =
    c.getMPG();
```

Assembly language:

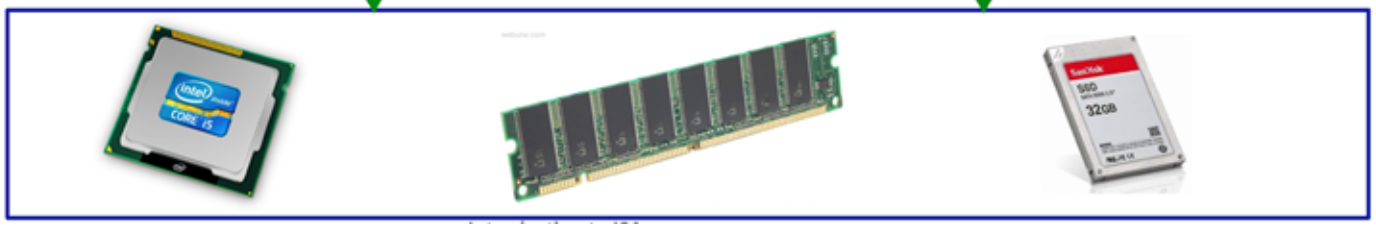
```
get_mpg:
    pushq   %rbp
    movq    %rsp, %rbp
    ...
    popq    %rbp
    ret
```

Machine code:

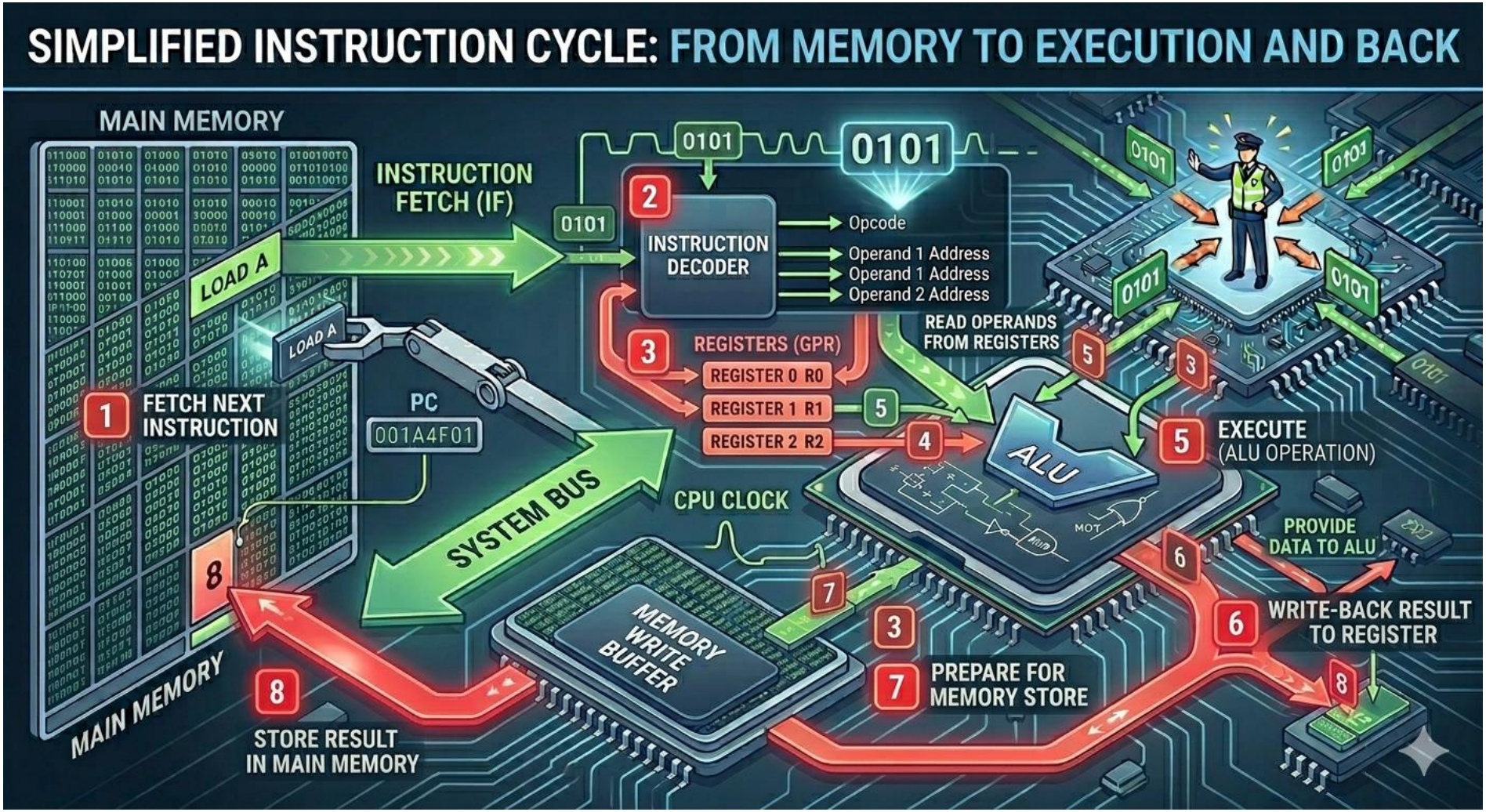
```
0111010000011000
100011010000010000000010
1000100111000010
110000011111101000011111
```

OS:

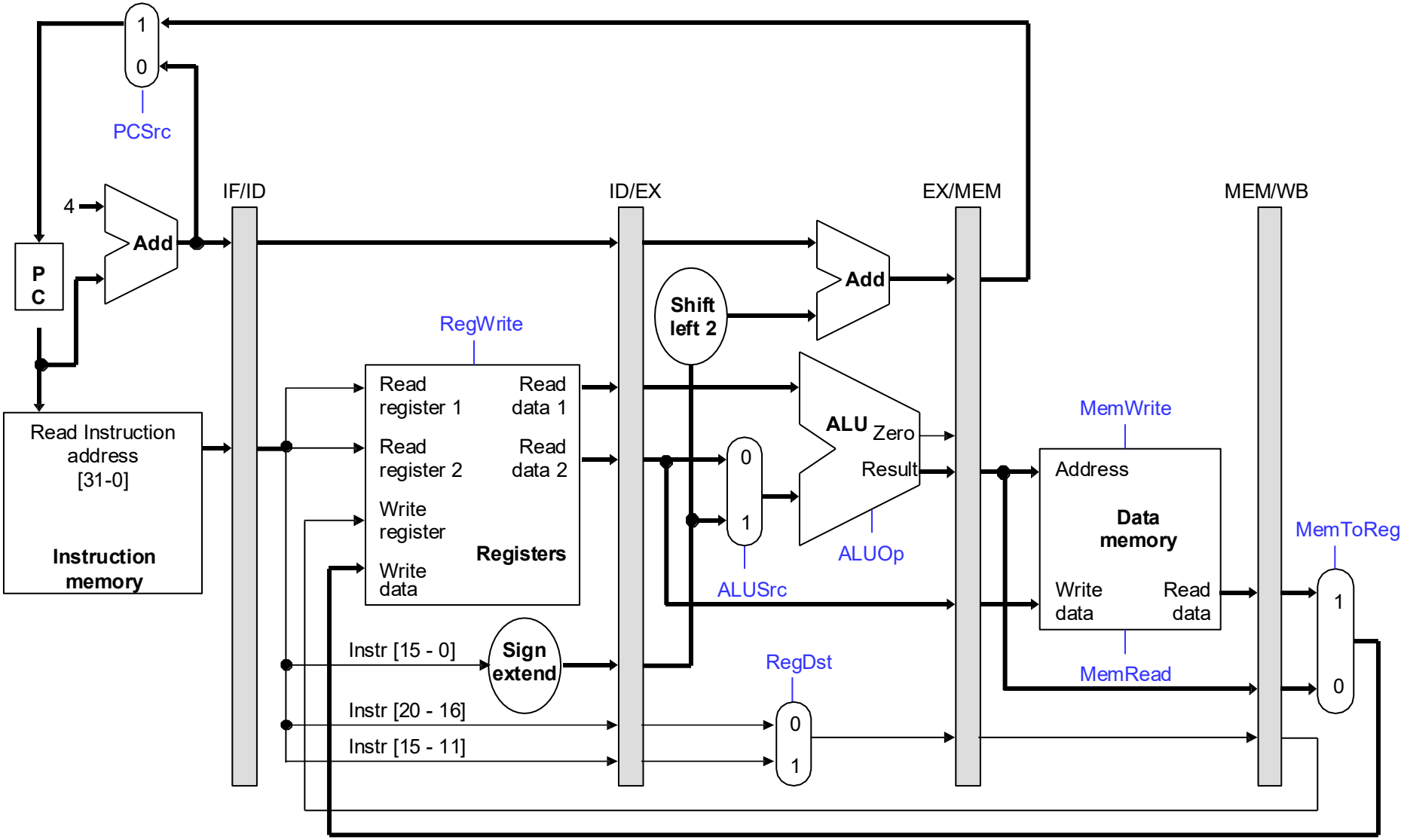
Computer system:



Executing An Instruction



A Simple Processor Design



Basics of Processors

Q: *How does a processor execute machine code?*

- Types of Instruction Set Architecture (ISA) commands to manipulate register contents:
  - ***Memory access*: load** (copy bytes from DRAM address to register); **store** (reverse); put constant
  - ***Arithmetic & logic*** on data items in registers: add/multiply/etc.; bitwise ops; compare, etc.
  - ***Control flow*** (branch, call, etc.)
- **Caches:** Small local memory to buffer instructions/data

Processor Performance

***Q:** How fast can a processor process a program?*

- Modern CPUs can run millions of instructions per second!
  - ISA tells us #**clock cycles** each instruction needs
  - CPU's **clock rate** lets us convert that to runtime (ns)
- Alas, most programs do not keep CPU always busy
  - Memory access commands **stall** the processor; Arithmetic & Logic Unit and Control Unit are *idle* during memory-register transfer
  - Worse, data may not be in DRAM—wait for disk I/O!
  - So, actual *execution runtime* of program may be orders of magnitude higher than what clock rate calculation suggests

Key Principle: Optimizing access to main memory and use of processor cache is critical for processor performance!

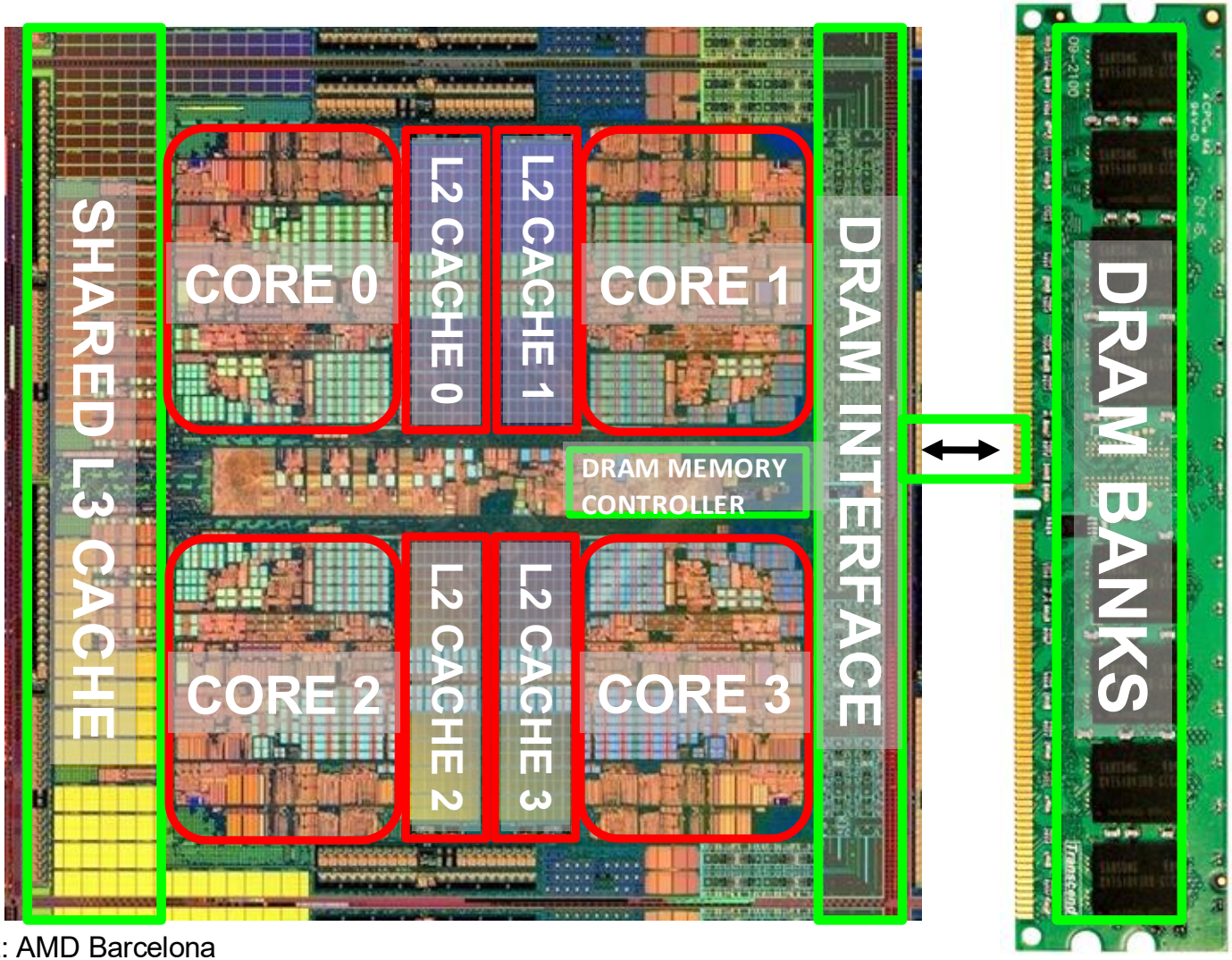
Q: *But why bother learning such low-level computer sciencey stuff in Data Science?*

Crossing Abstraction layers

- **As long as** everything goes well, not knowing what happens in the underlying level (or above) is not a problem.
- What if
  - The program you wrote is running slow?
  - The program you wrote does not run correctly?
  - The program you wrote consumes too much energy?
- What if
  - The hardware you designed is too hard to program?
  - The hardware you designed is too slow because it does not provide the right primitives to the software?
- One goal of this course is to understand how a processor works underneath the software layer and how decisions made in hardware affect the software/programmer

An Example: Multi-Core Systems

Multi-Core Chip



*Die photo credit: AMD Barcelona

A Memory Performance Hog

```
for (j=0; j<N; j++) {
    index = j*linesize; //linesize=1
    A[index] = B[index]; streaming
    …
}
```

```
for (j=0; j<N; j++) {
    index = rand()%N;
    A[index] = B[index]; random
    …
}
```

STREAM

- Sequential memory access
- Very high row buffer locality (96% hit rate)
- Memory intensive

RANDOM

- Random memory access
- Very low row buffer locality (3% hit rate)
- Similarly memory intensive

Moscibroda and Mutlu, "Memory Performance Attacks," USENIX Security 2007.

Disparity In Slowdown

- On moving to a multi-core arch, the two programs suffered from a disparity in slowdowns.
- Can you figure out why if you do not know how the processor executes the programs?
- Can you fix the problem without knowing what is happening “underneath”?

The Root Cause

- Multiple threads share the DRAM controller
- DRAM controllers are designed to maximize DRAM throughput
- DRAM scheduling policies are thread-unfair and prioritize threads with high row buffer locality

Thought Exercise

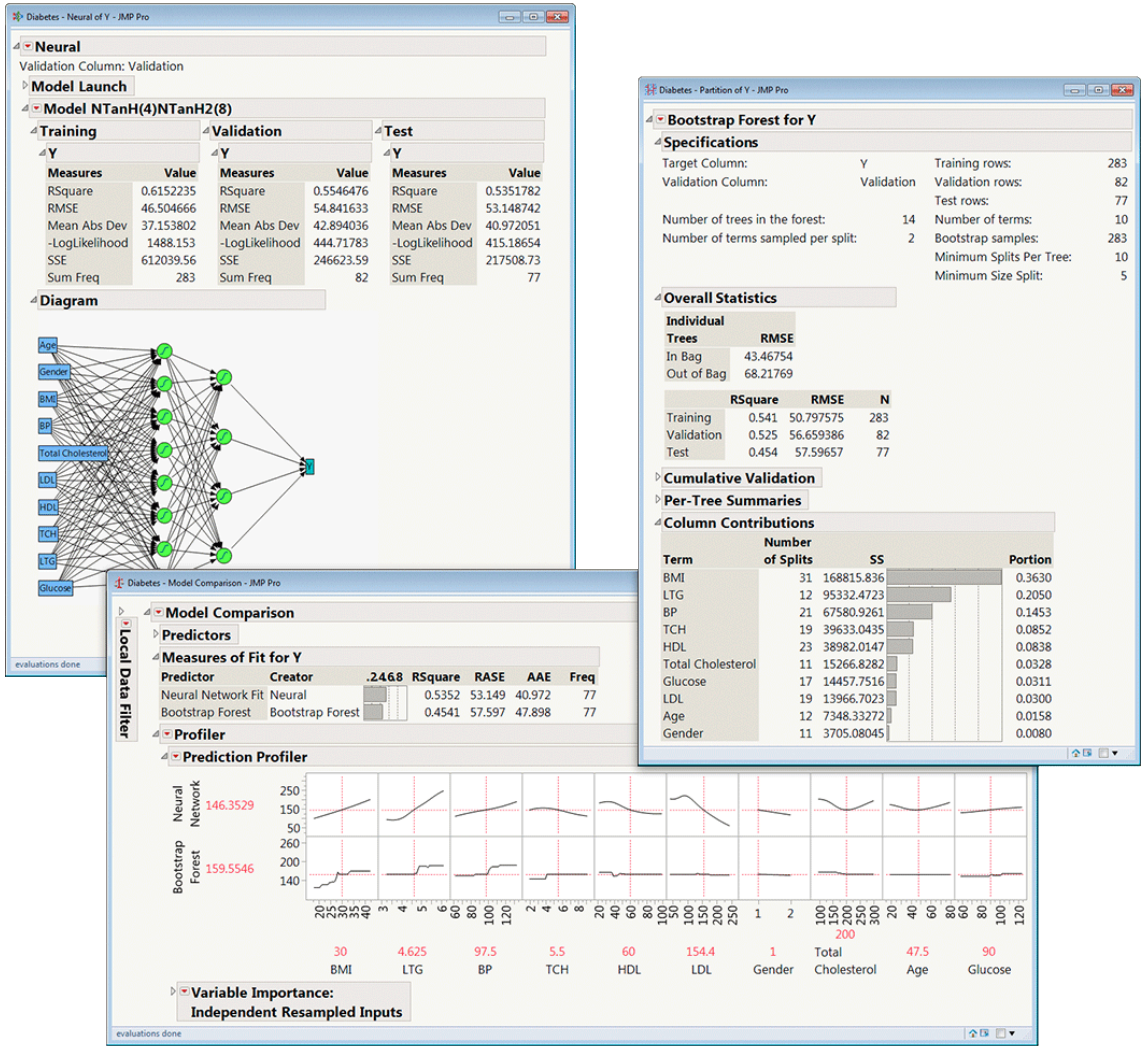
- You are thinking of buying a computer and there are two options:
  - CPU of 2.8 GHz
  - CPU of 3.6 GHz

You will use this machine for big data science like we discussed in the previous class. Both cost the same. Which one will you buy?

Clock speed tells you how fast the clock ticks.
It does NOT tell you how much work gets done each tick.

Luxury of “Statisticians”/“Analysts” of Yore

- **Methods:** Sufficed to learn just math/stats, maybe some SQL
- **Types:** Mostly tabular (relational), maybe some time series
- **Scale:** Mostly small (KBs to few GBs)
- **Tools:** Simple GUIs for both analysis and deployment; maybe an R-like console



https://www.jmp.com/en_au/offers/jmp-pro-for-academic-research.html
https://www.technologymagazine.com/data-and-data-analytics/sas-tops-worldwide-advanced-and-predictive-analytics-market-share

Reality of Today's "Data Scientists"

"Unicorn" status (highly sought-after, but scarce) of all-in-one team players with skills like:

- handling computationally any size of datasets
- possessing statistical prowess and modeling skills
- understanding and programming with databases
- solving problems; visualize, communicate well
- extracting business value from data
- knowing how to use AI to co-write code, build prompts & agents

Why bother with these in Data Science?

- Basics of Computer Organization
  - Digital Representation of Data
  - Processors and Memory Hierarchy
- Basics of Operating Systems
  - Process Management: Virtualization; Concurrency
  - Filesystem and Data Files
  - Main Memory Management
- Persistent Data Storage

You will face myriad and new data types

Compute hardware is ever-evolving

You will need to use new methods on evolving data file formats on clusters & cloud

Storage hardware is evolving fast

Topic 1 – Outline

- Basics of Computer Organization
  - Digital Representation of Data
  - Processors and Memory Hierarchy
- Basics of Operating Systems
  - Process Management: Virtualization; Concurrency
  - Filesystem and Data Files
  - Main Memory Management
- Persistent Data Storage